

Flowing with Confidence

Friso de Kruijf^{1,2*} Dario Coscia^{3,2†} Max Welling^{1,2} Erik Bekkers²

¹CuspAI ²AMLab, University of Amsterdam ³mathLab, SISSA

Abstract

Generative models can produce nonsensical text, unrealistic images, and unstable materials faster than simulation or human review can absorb; without per-sample confidence, trust erodes. Existing fixes run k ensembles or stochastic trajectories at $k \times$ compute, measuring variability between models, not model confidence. We propose Flow Matching with Confidence (FMwC). FMwC injects input-dependent multiplicative noise at selected layers, propagates its variance through the network in closed form, and integrates it along the ODE trajectory, yielding a per-sample confidence score at standard sampling cost. The score supports multiple uses: filtering improves image quality and thermodynamic stability of crystals; editing rewinds trajectories to the points where the model commits and redirects them; and adaptive stepping concentrates ODE compute where the flow is ambiguous. We find that the confidence score correlates with the magnitude of the divergence of the learned velocity field, which gives us a window to understand the generative process, opening up surgical forms of guidance that target the moments that matter, new sampling algorithms and interpretability of generative models.

1 Introduction

Generative models have become a central pillar of modern machine learning, producing samples across language, vision, and the natural sciences. Within this class, flow matching has emerged as a leading approach, training a deterministic velocity field by simulation-free regression and integrating it at inference to produce samples [Lipman et al., 2022]. Despite their broad success [Miller et al., 2024, Gat et al., 2024, Qin et al., 2024, Eijkelboom et al., 2024], state-of-the-art models still generate outputs that contain artefacts, and each sample arrives with no native indication of how reliable it is: a hallucinated citation, an anatomically broken figure, and a thermodynamically impossible crystal each emerge with the same implicit confidence. The cost of this missing signal compounds with deployment: in materials discovery, for instance, validating a single generated candidate can take days of compute or weeks of laboratory time, and indiscriminate generation wastes both. This motivates the central question of this work:

Can flow-matching models learn to know what they don't know?

This question echoes Nalisnick et al. [2019], who showed that the log-likelihoods of deep generative models cannot reliably distinguish in- from out-of-distribution inputs. We ask an orthogonal question on the generation side: not whether the model can flag anomalous inputs from its likelihood, but whether it can produce a per-sample reliability signal for the outputs it itself generates, a signal that does not require access to a likelihood at all.

Confidence estimation [Berry and Meger, 2023, Jazbec et al., 2025] offers a route to such a signal, yet existing approaches force a costly trade-off between training and inference budgets. Deep ensembles [Lakshminarayanan et al., 2017] train k models; MC-Dropout [Gal and Ghahramani,

*Correspondence to f.c.dekruijf@uva.nl.

†Work done in collaboration with the University of Amsterdam.

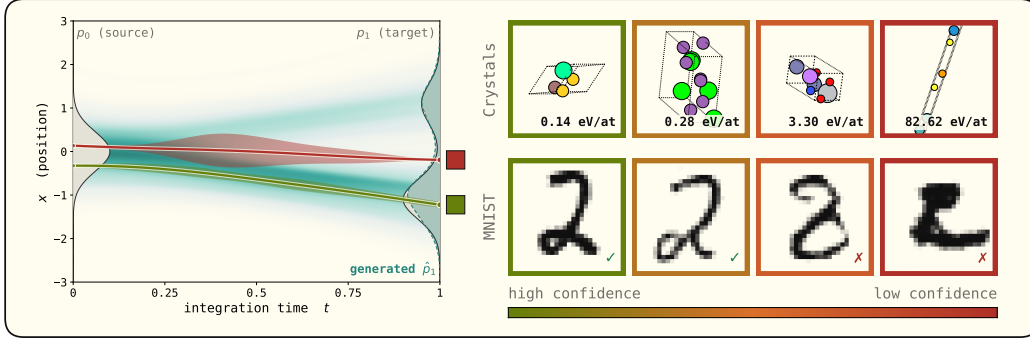


Figure 1: **Flow-Matching model with Confidence.** (Left) Variance is propagated alongside the sample as it is generated along the path $p_0 \rightarrow p_1$: well-placed samples (green) contract into a tight $\pm\sigma$ band, while misplaced ones (red) stay diffuse. (Right) Sorting by the resulting confidence score recovers quality across modalities—low-energy crystals and clean digits at high confidence, implausible structures and malformed digits at low confidence.

2016], SDE-style perturbation, and post-hoc last-layer Laplace methods [Jazbec et al., 2025] instead run k trajectories per output at inference. Beyond their cost, all of these report *disagreement between models or between runs* rather than a property of the single model that produced the output, leaving per-sample confidence either expensive, or unavailable (Table 1).

We propose **Flow Matching with Confidence (FMwC)**, a flow-matching model that produces a calibrated per-sample confidence score for every generated output, at the same training and inference cost as the deterministic baseline. The construction builds on variational dropout [Kingma et al., 2015] with input-dependent rates [Coscia et al., 2025b], and departs from prior variational-dropout work by propagating variance through the entire ODE trajectory rather than through the network at a single time step; that is what turns per-layer learned noise into a trajectory-level confidence score. The resulting score is a within-model, geometric signal that reflects how the learned velocity field responds to perturbations of its own hidden states, distinct from the between-model disagreement that ensembles and MC-Dropout measure. The main contributions of this paper are:

1. **A single-pass confidence estimator for flow matching.** We introduce Flow Matching with Confidence (FMwC), a flow-matching model that emits a per-sample, trajectory-level confidence score alongside every generated sample, from a single deterministic forward integration as the standard flow-matching baseline. The score is obtained by analytically propagating learned, input-dependent layer-wise noise through the ODE, yielding a closed-form trajectory variance with no Monte Carlo sampling and no auxiliary networks.
2. **Confidence enables practical, training-free improvements at inference.** Filtering by FMwC monotonically improves sample quality over the deterministic baseline. Second, the temporal variance peak identifies the exact moment the model commits to a sample enabling targeted, constraint-respecting edits without retraining, while confidence score adaptive integration recovers quality at low step budgets where uniform stepping degrades sharply. The same signal supports zero-shot quality selection in confidence-conditioned generation, and serves as a single-pass proxy for flow diagnostics that would otherwise require multiple integrations.
3. **Confidence reflects the intrinsic geometry of the learned flow.** We show that FMwC’s single-pass score tracks the local divergence of the learned velocity field. This establishes the confidence score as a within-model geometric quantity, a property of the single trained flow rather than a proxy for inter-model disagreement.

2 Related Work

Flow matching has a unique target with intrinsically delicate geometry. Flow matching trains a velocity field by simulation-free regression against a conditional target [Lipman et al., 2022], and has been extended to general geometries [Chen and Lipman, 2024] and graph-structured data [Eijkelboom et al., 2024]. Because the conditional and marginal flow-matching gradients coincide,

Table 1: Comparison of uncertainty quantification approaches for flow matching. FMwC is the only method that provides per-sample confidence at the cost of a single deterministic forward pass, matching the inference budget of standard flow matching.

Method	Trained models	Sampled trajectories	Confidence
FM	1	1	✗
MC-Dropout	1	k	✓
Ensemble	k	k	✓
FMwC (ours)	1	1	✓

the optimal solution is uniquely determined [Lipman et al., 2022]. It is a conditional expectation: at each intermediate state, the velocity averages over the conditional velocities of all endpoints reaching that state [Pooladian et al., 2023]. This averaging structure renders the optimal field geometrically delicate: it undergoes spontaneous symmetry-breaking transitions at critical times, where the trajectory’s posterior over endpoints contracts onto a single mode and the susceptibility of the velocity field diverges [Raya and Ambrogioni, 2023, Ambrogioni, 2024], with the local divergence of the field further governing how information concentrates along the trajectory [Stančević and Ambrogioni, 2026]. A finite-capacity network trained on finite data approximates such a field imperfectly, with the approximation expected to be least reliable in regions of intrinsic instability. The standard flow-matching architecture, however, produces a deterministic velocity at inference and exposes no per-sample signal of where the learned flow is trustworthy.

Per-sample reliability methods at inference are expensive or measure the wrong quantity.

Likelihood is not a per-sample reliability signal for deep generative models [Nalisnick et al., 2019, Kamkari et al., 2024], motivating methods that produce one explicitly. Deep ensembles [Lakshminarayanan et al., 2017] train k independent networks; at inference, each generated sample requires k separate trajectory integrations, scaling the per-sample cost by k at every Euler step. MC-Dropout [Gal and Ghahramani, 2016] reuses a single trained model but still requires k stochastic trajectories per output. The post-hoc last-layer Laplace family [Jazbec et al., 2025] draws k weight realisations and runs k full sampling trajectories per output. Each of these methods reports disagreement between models, between runs, or between externally embedded outputs, and demanding significantly more compute than standard flow matching (Table 1). A reliability signal at the inference cost of standard flow matching has not been demonstrated yet.

Toward efficient uncertainty estimation. A separate line of work focuses on efficient uncertainty estimation. Variational Adaptive Dropout (VAD) learns a small auxiliary network to predict the stochastic perturbation scale for each network’s weights, so the posterior broadens where the input is ambiguous and tightens where it is not [Coscia et al., 2025b,a]. Crucially, under VAD the predictive distribution at each linear layer is Gaussian and available in closed form, but stacking layers with nonlinearities breaks this closed form, needing multiple trajectory samples to obtain moments estimates. Sampling-free moment propagation closes exactly this gap: it pushes moments through pointwise nonlinearities analytically, by local linearisation [Wang and Manning, 2013] or by quadrature [Postels et al., 2019], recovering a closed-form predictive distribution across the full depth of the network in a single pass. Combining input-dependent variational posteriors with closed-form moment propagation through every layer has not, to our knowledge, been carried into a trajectory-valued generative ODE.

3 Method

FMwC is a single-pass approach for equipping any flow-matching model with a per-sample confidence signal. It treats the network’s weights stochastically by injecting input-dependent Gaussian noise, and propagates the resulting variance analytically through the ODE alongside the deterministic trajectory. In the rest of this section, we formalise FMwC as a variational posterior over velocity fields, derive the variance recursion through the network and the integrator, and define scalar readouts that summarise the trajectory variance and can be used as confidence scores.

3.1 Flow Matching with Confidence

Flow matching. Flow matching [Lipman et al., 2022] learns a continuous-time transformation from a base distribution p_0 to a target density $p_1 := p_{\text{data}}$ by training a time-dependent velocity field $v_{t,\theta}(\mathbf{x}_t)$ that transports p_0 to p_1 along a prescribed probability path, where θ are the trainable network parameters. At inference, samples are drawn by integrating the learned field from $t=0$ to $t=1$. Standard flow matching is silent about the reliability of each step: the network outputs a single value $\mu_t := v_{t,\theta}(\mathbf{x}_t)$, and the integrator advances on it without further information. The remainder of this section constructs the missing object: a per-step variance σ_t^2 that quantifies how trustworthy each velocity estimate is. We refer the reader to Appendix A.2 for a self-contained review of flow matching, including the conditional formulation and training objective.

From point estimate to posterior. Even though the conditional flow-matching target is uniquely determined, the field it specifies is geometrically delicate (Sec. 2): symmetry-breaking transitions and finite-data fitting noise make the velocity locally unreliable in characteristic regions, and a single point estimate carries no signal of where it is trustworthy. We therefore treat the network weights ω as random and learn an approximate posterior $q_\psi(\omega)$ by maximising the weight-space ELBO

$$\text{ELBO}(\psi) = \mathbb{E}_{\omega \sim q_\psi} [\log \mathcal{L}(\mathcal{D} \mid \omega)] - \text{KL}[q_\psi(\omega) \parallel \pi(\omega)], \quad (1)$$

with flow-matching log-likelihood

$$\log \mathcal{L}(\mathcal{D} \mid \omega) = - \mathbb{E}_{t \sim \mathcal{U}(0,1), \mathbf{x}_t \sim p_t} [\|u_t(\mathbf{x}_t) - v_{t,\omega}(\mathbf{x}_t)\|^2]. \quad (2)$$

Here ψ denotes all variational parameters. Eq. (1) is obtained by inducing $q_\psi(v_t)$ as the pushforward of $q_\psi(\omega)$ through the network parameterization $\omega \mapsto v_{t,\omega}$, with a matching pushforward prior; we work directly in weight space and defer the formal functional derivation to App. B.

This formulation gives us two useful properties. First, the simulation-free conditional flow-matching estimator of Lipman et al. [2022] extends to the variational setting: the gradient of the expected log-likelihood under q_ψ coincides with the gradient of the standard CFM regression loss evaluated on weight samples (Obs. B.2), so we inherit a tractable, simulation-free training objective at no extra cost. Second, the posterior-mean velocity field $\mu_t(\mathbf{x}) := \mathbb{E}_{\omega \sim q_\psi} [v_{t,\omega}(\mathbf{x})]$ still satisfies the continuity equation (Obs. B.1), so Bayesian model averaging produces a valid generative flow rather than a heuristic combination of vector fields. Together, these properties mean the variational treatment preserves both the training procedure (up to a KL regularizer) and the generative semantics of standard flow matching, while equipping the model with a posterior over velocities from which a per-sample confidence signal can be derived.

State-dependent posterior via VAD. The construction so far leaves the variational family q_ψ unspecified. A standard mean-field Gaussian would suffice for tractability, but it would assign the same posterior spread everywhere in the trajectory and, by Obs. B.1, average over a fixed set of admissible vector fields regardless of where the flow is locally reliable. The pathology we are trying to detect, however, is fundamentally local: critical times where the trajectory’s posterior over endpoints contracts onto a single mode are exactly where the velocity becomes most sensitive (Sec. 2), and these regions are not known in advance. We therefore want a posterior whose spread is itself a function of the state $\mathbf{x}$ and time t along the trajectory.

We parameterise q_ψ using variational adaptive dropout (VAD) [Coscia et al., 2025b], which makes the per-weight perturbation depend on the input. Each weight ω_l in layer l has the Gaussian posterior

$$q_\psi(\omega_l \mid \mathbf{x}, t) = \mathcal{N}(\theta_l, \alpha_l(\mathbf{x}, t)^2 \theta_l^2), \quad (3)$$

and sampled via the local-reparametrization trick, see App. A.3.3 for more details on VAD. The dropout scale $\alpha_l(\mathbf{x}, t)$ is the output of a small per-layer inference network E_γ ; the trainable parameters of FMwC are therefore $\psi = \{\theta, \gamma\}$, with θ the main model weights and γ the inference network weights. The role of the inference network is to broaden the posterior at points of geometric ambiguity and tighten it elsewhere, learned without supervision: the only training signal it receives is the ELBO itself, in which the KL term reduces to a closed-form expression in α_l alone (Eq. 19) and acts as a regulariser on the dropout rates beyond the standard flow-matching loss.

3.2 Variance Propagation

VAD as introduced in §3.1 gives a Gaussian posterior over weights at each linear layer; under the local reparameterisation [Kingma et al., 2015], this induces a Gaussian pre-activation in closed form. Stacking layers with nonlinearities generally breaks this closed form. Sampling-free moment propagation closes this gap within a single forward pass [Wang and Manning, 2013, Postels et al., 2019]; FMwC carries this further by propagating moments through both the network *and* the ODE trajectory in one integration: at every ODE step, we compute the posterior-mean velocity μ_t together with its variance σ_t^2 .

Through the network. Under the local reparameterisation, the pre-activation at each linear layer l is Gaussian with closed-form mean and variance, so the pair³ (μ_l, σ_l^2) propagates layer-to-layer without sampling [Molchanov et al., 2017]:

$$\mu_{l+1} = \theta_l \mu_l + b_l, \quad \sigma_{l+1}^2 = \alpha_l (\theta_l^{\odot 2} \mu_l^{\odot 2}) + (\alpha_l + 1) (\theta_l^{\odot 2} \sigma_l^2), \quad (4)$$

where (μ_l, σ_l^2) denotes the post-activation moments entering layer l and $(\mu_{l+1}, \sigma_{l+1}^2)$ the pre-activation moments leaving it; $\theta_l^{\odot 2}$ denotes elementwise square, and **blue** = mean, **red** = variance, **orange** = learned noise scale. The recursion implies that variance grows multiplicatively with depth, making multi-layer VAD propagate uncertainty through the network. Finally, pointwise nonlinearities ϕ require computing $\mathbb{E}[\phi(z)]$ and $\text{Var}[\phi(z)]$ for $z \sim \mathcal{N}(\mu_l, \sigma_l^2)$; we use 10-node Gauss–Hermite quadrature [Postels et al., 2019], with first- and second-order Taylor linearisation [Wang and Manning, 2013] as a near-identical alternative at the depths used here (Tab. 10).

Through the ODE. At each Euler step the network emits the pair (μ_t, σ_t^2) , where μ_t is the posterior-mean velocity defined above. Advancing the position along $\mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mu_t \Delta t$ therefore integrates the Bayesian model average flow, which Obs. B.1 guarantees is itself a valid generator. The variance σ_t^2 is recorded at every step. This is the design choice that distinguishes FMwC from MC-Dropout and Laplace methods: rather than sampling k velocity fields and integrating k trajectories, we integrate the posterior-mean trajectory and track the variance analytically along it, at approximately the same cost as deterministic flow matching. The output of a single forward integration is therefore not a scalar uncertainty but a *trajectory* of velocity variance with non-trivial temporal structure (Fig. 1, left): variance can fall as the flow commits to a mode, or persist as the flow fails to.

3.3 Reading the Trajectory

For filtering, the variance trajectory $\{\sigma_t^2\}_{t=0}^T$ (the full per-step sequence emitted along the integration) must be aggregated into a per-sample scalar that ranks samples for retention; other downstream uses can read the trajectory directly (Sec. 4). Three label-free aggregations come directly from the trajectory’s geometry. *Endpoint confidence* reads off σ_T^2 at the final step,

$$c_{\text{ep}}(\mathbf{x}_0) = \|\sigma_T\|_2; \quad (5)$$

integrated confidence sums per-step velocity std along the trajectory, $c_{\text{int}}(\mathbf{x}_0) = \sum_t \|\sqrt{\sigma_t^2}\|_2 \Delta t$; and the *temporal confidence ratio* contrasts late- with early-window variance,

$$c_{\text{tc}}(\mathbf{x}_0) = \frac{\sum_{t > \tau_\ell} \|\sqrt{\sigma_t^2}\|_2}{\sum_{t < \tau_e} \|\sqrt{\sigma_t^2}\|_2}, \quad (6)$$

which lifts when converging samples shrink late variance while stuck samples do not. The full sequence of variance along the trajectory is also a useful signal to link to specific downstream outcomes when labels are available: given a small held-out set with an outcome flag (a downstream-quality threshold, a classifier-failure label, a stability target), we can map the sequence to that outcome directly via a learned head $c_\phi(\mathbf{x}_0) = \phi_\eta(\sigma_0^2, \dots, \sigma_T^2)$, fit with L1-regularised logistic regression or gradient boosting; this turns $\{\sigma_t^2\}$ into a task-specific score at little extra inference cost. Cross-method baselines re-cast the same confidence by resampling instead: MC-dropout and deep ensembles draw k trajectories from $\mathbf{x}_0$ and report the endpoint dispersion $c_{\text{disp}} = \|\text{std}_k(\mathbf{x}_1^{(i)})\|_2$, at $k \times$ trajectory cost.

³We drop the time index on internal layer moments for clarity, writing (μ_l, σ_l^2) for the moments at layer l at fixed time t . The subscript t is reserved for the network’s output, i.e. (μ_t, σ_t^2) are the moments at the final layer.

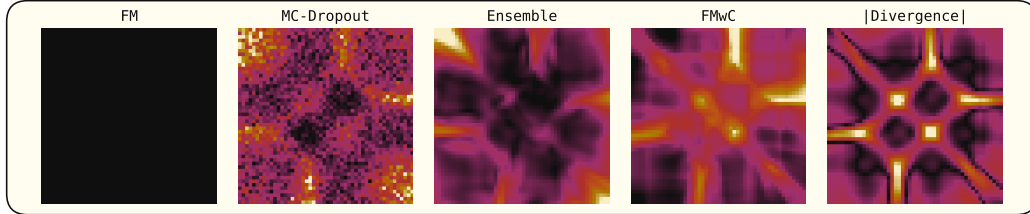


Figure 2: σ_t^2 recovers the divergence structure of the learned velocity field at bifurcation time. Spatial fields at $t=0.6$ on the Checkerboard, where $|\nabla \cdot v_{t,\theta}|$ (right) is available analytically. Quantitative correlation across modalities in Sec. 4.

3.4 Where and When Does Uncertainty Arise?

The construction above is mechanically motivated, but the reason the resulting σ_t^2 is informative is geometric. The optimal flow-matching velocity is a conditional expectation over endpoints (Sec. 2), and at critical times this average undergoes spontaneous symmetry-breaking: the endpoint posterior contracts onto a single mode, the field’s susceptibility diverges, and the local divergence of the velocity governs how information concentrates [Raya and Ambrogioni, 2023, Ambrogioni, 2024, Stančević and Ambrogioni, 2026]. At these bifurcation points the regression target is an average over multiple still-plausible continuations, so any single-valued prediction is irreducibly uncertain — and this is the regime where finite-capacity ELBO training also has the largest residual error to explain.

The inference network picks up on this without supervision. Maximising the ELBO trades reconstruction error against KL: tighter posteriors (smaller α_l) reduce the KL but inflate reconstruction, and the optimum broadens $\alpha_l(x, t)$ exactly where the velocity is hardest to fit, i.e. the bifurcation regions. Eq. 4 then amplifies this broadening multiplicatively with depth, leaving bifurcation-point inputs with large σ_t^2 . Figure 2 shows this on the Checkerboard distribution, where $|\nabla \cdot v_{t,\theta}|$ is available analytically: the single-pass σ_t reproduces its structure at the cost of a mere deterministic forward pass. Quantitative correspondence across modalities is reported in Sec. 4.

4 Experiments

We demonstrate the effectiveness of FMwC by applying it across domains where generative modelling plays a central role. First, as illustrative examples, we apply FMwC to a 2D Checkerboard and to class-conditional MNIST generation under standard Euclidean flow matching. We then turn to a more challenging setting and apply FMwC to de novo inorganic crystal generation under Riemannian flow matching, where it must operate on a non-Euclidean manifold. We first verify that standard generation under FMwC preserves sample quality, then exercise the confidence score in three downstream uses: filtering low-confidence samples, editing trajectories at the timesteps where they lock onto a mode, and adapting ODE step size to where the flow is ambiguous. Finally, we show that σ_t^2 indeed correlates with the magnitude of the learned velocity field’s divergence, with the correlation strengthening as the geometry of the flow becomes richer. For additional details on datasets, model hyperparameters and metrics we refer to App. C.

Table 2: **Standard generation under FMwC matches or improves FM on every modality, with negligible inference overhead at scale.** Quality columns: one headline metric per modality (lower is better in all three). Cost columns: forward-pass FLOPs ratio relative to FM, measured. Bold marks the best per column.

Method	Quality ($\downarrow$)			Inference cost vs FM ($\downarrow$)		
	Checkerboard mispl %	MNIST FID	Crystals $\bar{e}_{a.h.}$	Checkerboard	MNIST	Crystals
FM	5.9	5.17	0.21	1×	1×	1×
MC-Dropout ($k=5$)	7.5	4.25	—	5×	5×	—
Ensemble ($k=5$)	5.9	4.35	—	5×	5×	—
FMwC	4.6	4.22	0.21	3.22×	1.003×	1.08×

Standard generation under FMwC preserves and improves sample quality. Table 2 reports sample quality across the three datasets. FMwC matches or improves FM’s quality metric on every dataset, and notably improves over FM on Checkerboard and class-conditional MNIST, showing that learning the confidence score can also improve sample quality. Inference cost scales favourably with backbone size. VAD adds a roughly fixed cost per layer, so on the tiny Checkerboard backbone this overhead is significant, but on larger backbones like MNIST and FlowMM the backbone itself dominates and the overhead becomes negligible. Finally, multi-trajectory baselines are absent on Crystals because deep ensembles would require $5\times$ retraining of FlowMM and MC-dropout would require $5\times$ stochastic trajectories per generated structure, both prohibitive at the throughput LeMat GenBench [Betala et al., 2025] evaluates against.

Table 3: **FMwC trajectories carry filtering signal across modalities at single-trajectory cost.** AUPRC ($\uparrow$) of confidence-guided retention on Checkerboard misplacement, MNIST classifier failure, and Crystal $e_{\text{above hull}} < 0.1$ eV/atom. The FM row reports the positive-class rate (random-retention floor); meaningful comparison is lift over it. Row tints denote scoring family: **blue** = unsupervised aggregations of the FMwC variance trajectory, **green** = supervised heads on engineered features.

Method	Scoring	Toy AUPRC $\uparrow$	MNIST AUPRC $\uparrow$	Crystal AUPRC $\uparrow$	Traj. $\downarrow$
FM	random	0.06	0.04	0.03	1
MC-Dropout ($k=5$)	endpoint dispersion	0.60	0.08	—	5
Ensemble ($k=5$)	endpoint dispersion	0.55	0.11	—	5
FMwC ($k=5$)	endpoint dispersion	0.58	0.04	0.03	5
FMwC	endpoint confidence	0.30	0.02	0.04	1
FMwC	integrated confidence	0.33	0.02	0.04	1
FMwC	temporal ratio	0.55	0.04	0.03	1
FMwC	learned (L1-LR)	0.65	0.09	0.06	1
FMwC	learned (GB)	0.83	0.09	0.06	1

Filtering low-confidence samples reaches ensemble-grade lift at single-trajectory cost. Table 3 reports filtering AUPRC on the three modalities. The FM row is the random-retention floor (it equals the positive-class rate); what matters is lift above that floor. Two findings stand out. First, on Checkerboard the temporal aggregation of σ_t^2 reaches 0.55 AUPRC from a single trajectory, matching the $k=5$ MC-Dropout and Ensemble baselines at one fifth of the inference cost. Second, a small supervised head trained on the variance trajectory is the best single-trajectory filter on every modality, 0.83 on Checkerboard, 0.09 on MNIST, 0.06 on Crystals, still using a single trajectory. The MNIST and Crystals numbers are modest in absolute terms because the targets are hard (a classifier disagreeing with the conditioned digit; an unrelaxed UMA single-point predicting $e_{\text{above hull}} < 0.1$ eV/atom), but in every case the variance trajectory carries information the FM baseline does not.

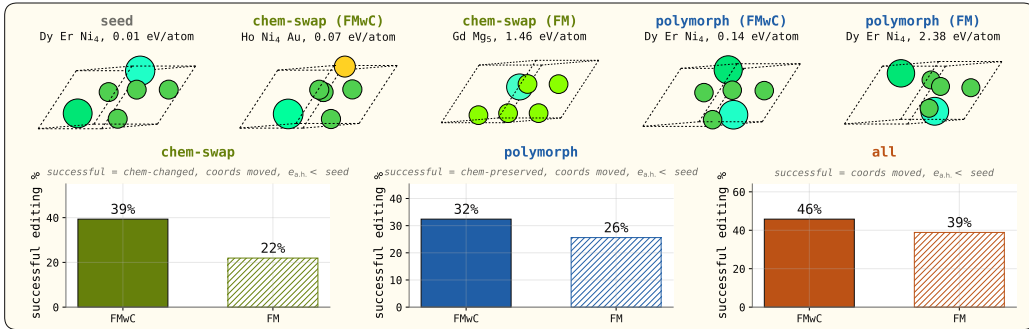


Figure 3: **FMwC’s per-channel t^* targets a specific bifurcation; FM at random t misses it.** (a) Seed Dy Er Ni₄ (on the hull) under chem-swap (atoms-only at t_{atom}^*) and polymorph (coords-only at t_{coord}^*): FMwC’s edits stay near the hull, FM’s drift off. (b–d) Per-mode success rate over 50 seeds $\times$ 20 replicates; FMwC ($t=t^*$, solid) vs FM (uniform random t , hatched), at matched channel mask and σ . Success requires the per-mode chemistry intent (chem-changed for chem-swap, chem-preserved for polymorph, either for all-channels), coord-RMSD > 0.10 , and $e_{\text{a.h.}}$ below the seed’s.

FMwC enables controlled editing of crystals without retraining. The variance trajectory exposes *when* the model commits to a structure. To edit a generated sample we (i) pick a perturbation timestep t^* and (ii) inject Gaussian noise into the channel being edited at t^* , then re-integrate the deterministic flow to $t=1$ with all channels free to update. The channel is one of atoms, coordinates, or the full latent; FMwC picks t^* as the argmax of σ_t^2 in that same channel, while FM picks t^* uniformly at random. Figure 3 reports successful editing rate on FlowMM under three modes that pair an injection channel with a success criterion: *chem-swap* injects atom-channel noise and counts a success when the chemistry changes; *polymorph* injects coord-channel noise and counts a success when the chemistry is preserved; *all-channels* injects into the full latent and counts a success regardless of chemistry. All three additionally require the structure to have moved (periodic boundary aware coord-RMSD > 0.10) and $e_{a.h.}$ to fall below the seed’s. At matched channel mask and noise budget, FMwC produces 6–17 percentage points more successful edits than FM, with the gap widest on the constraint-respecting modes and narrowest on *all-channels*, the predicted pattern: locating the critical timestep matters most when the edit must respect a constraint. These experiments confirm empirically that the high- σ_t^2 regions of the trajectory coincide with critical decision points: editing at t^* reroutes the sample, while editing away from it either dissolves the perturbation (early t) or leaves no time to react (late t).

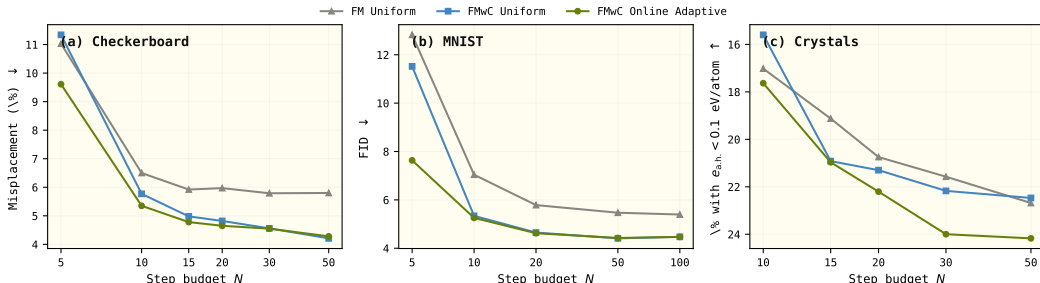


Figure 4: **A single per-sample adaptive-stepping signal, $\sigma_t^2(t)$, generalises across modalities.** Each panel plots the application’s headline quality metric against integrator step budget N , oriented so down-and-right is better (Crystal y-axis inverted). Methods: FM Uniform (grey), FMwC Uniform (blue), and FMwC Online Adaptive (green), with the controller chosen per modality.

Adaptive stepping reallocates compute to the timesteps where the flow is least decided. The variance trajectory suggests an obvious use: allocate integrator steps where the flow is least decided to improve sample quality. We instantiate this with per-sample controllers on σ_t^2 at a fixed number of forward evaluation (NFE) budget. A naive controller – an EMA on the trajectory norm of σ_t^2 – improves Checkerboard and MNIST but degrades the Riemannian crystal flow, concentrating compute on early-trajectory variance peaks that do not govern crystal quality. We therefore propose FMwC Online Adaptive, a per-component late-only damping rule that allocates extra steps only to the window in which the per-component variance ratio is rising. The full controller comparison is deferred to Appendix D.6.

Figure 4 shows that Online Adaptive improves over FM Uniform at every step budget across all three modalities. The gains are largest in the low-budget regime where uniform schedules are most wasteful: at $N=5$, Online Adaptive reduces Checkerboard misplacement from 11.0% to 9.6% and MNIST FID from 12.7 to 7.6. On Crystals, where the geometry is non-Euclidean and the EMA controller fails, Online Adaptive improves post-relaxation Meta% over FM Uniform at every $N \geq 10$ and matches at $N=30$ the quality FM Uniform attains at $N=50$.

σ_t^2 correlates with the magnitude of the learned-velocity-field divergence on every setting. Section 3.4 argues that σ_t^2 should peak at the bifurcation points where the divergence of the learned velocity field also peaks. We test this empirically. Since computing the exact divergence at scale is infeasible, we benchmark against a high- K Hutchinson stochastic-trace estimator of mean divergence and treat it as the reference signal. Our results, summarized in Figure 5, support the prediction in two ways. First, a single forward pass of FMwC matches the rank correlation against the high- K reference that Hutchinson reaches with $K \approx 13$ (Spearman) and $K \approx 7$ (Pearson) probes, the divergence signal is recovered at a fraction of the cost.

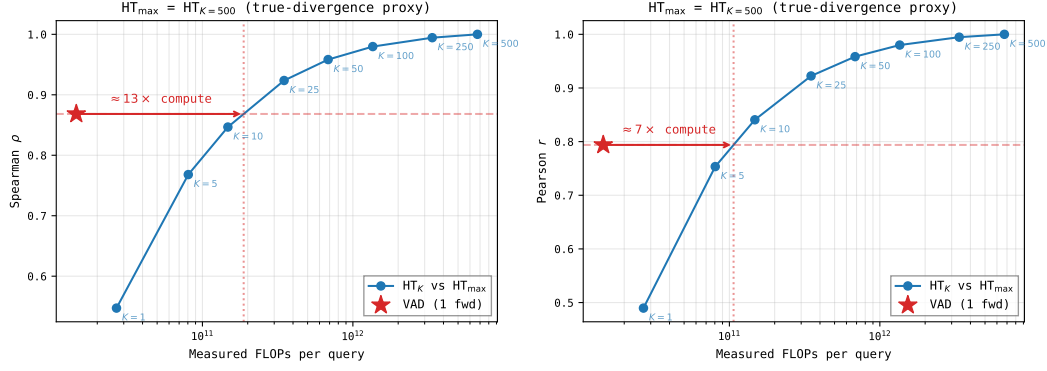


Figure 5: **On the Crystals model, one forward pass of FMwC matches Hutchinson’s divergence.** Spearman and Pearson correlation against the high- K Hutchinson reference vs. measured FLOPs per query, on the FlowMM backbone; sweep over $K \in \{1, \dots, 500\}$.

Second, the correspondence is universal and geometry-graded: Table 4 reports Pearson and Spearman correlations across all three modalities, in both per-step and integrated regimes. Integrated correlations are uniformly higher, as integration averages out timestep-local noise, and the correlation strengthens monotonically with the geometric richness of the flow, weakest on Checkerboard, stronger on MNIST, strongest on the Riemannian crystal flow.

Table 4: σ_t^2 **tracks the learned-velocity divergence magnitude** $|\nabla \cdot v_{t,\theta}|$ **across all three settings.** Pearson r (linear) and Spearman ρ (rank) reported in two regimes: per-step (correlation at each ODE timestep, averaged $\pm$ std) and integrated (correlation between trajectory-summed quantities, one scalar per sample). Single-method by construction: ensembles have no within-model analogue of σ_t^2 .

Application	Pearson $r(\sigma_t^2, \nabla \cdot v_{t,\theta}) \uparrow$		Spearman $\rho(\sigma_t^2, \nabla \cdot v_{t,\theta}) \uparrow$	
	Per-step (mean $\pm$ std)	Integrated	Per-step (mean $\pm$ std)	Integrated
Checkerboard	0.42 ± 0.26	0.71	0.36 ± 0.29	0.76
MNIST	0.46 ± 0.18	0.57	0.44 ± 0.17	0.58
Crystals	0.79 ± 0.09	0.93	0.87 ± 0.04	0.96

5 Conclusion

We presented Flow Matching with Confidence (FMwC), which turns flow matching from a generator of samples into a generator of samples paired with a measure of how much to trust them. By learning an input-dependent posterior over the velocity field’s weights, FMwC injects noise that regularises training and yields a confidence score at generation time, propagated analytically through the ODE in a single deterministic forward pass at the same cost as standard flow matching. Across 2D density estimation, image generation, and de novo inorganic crystal generation, FMwC preserves baseline sample quality on every modality, confirming that confidence training does not come at the expense of generation fidelity. Beyond the score itself, we show that the variance trajectory unlocks a range of downstream uses at no additional training or inference cost. Filtering by confidence improves sample quality across modalities, the variance peak exposes when the model commits to a structure and enables targeted, constraint-respecting edits without retraining, the variance trajectory drives a per-sample adaptive stepping rule that recovers high quality samples at low step budgets, and it also tracks the magnitude of the learned velocity field’s divergence. These results, together with the open questions and limitations discussed in Appendix E, suggest that FMwC makes confidence-aware generation a practical default for scientific and safety-critical applications, where reliable samples matter as much as plausible ones.

6 Acknowledgements

We thank Johann Brehmer, Floor Eijkelboom, Fabio Grätz, Pim de Haan, Metod Jazbec, Victor Kawasaki-Borruat, Alessandro De Maria, Govert Verkes for the thoughtful and helpful discussions. We thank the CuspAI team for the supportive research environment and for the engineering platform that this work builds on. This publication is part of the project SIGN with file number VI.Vidi.233.220 of the research programme Vidi, which is (partly) financed by the Dutch Research Council (NWO) under the grant <https://doi.org/10.61686/PKQGZ71565>.

References

- Luca Ambrogioni. The statistical thermodynamics of generative diffusion models: Phase transitions, symmetry breaking and critical instability. *arXiv preprint arXiv:2310.17467*, 2024.
- Thomas Bayes. An Essay towards solving a Problem in the Doctrine of Chances. By the late Rev. Mr. Bayes, FRS communicated by Mr. Price, in a letter to John Canton, A.M.F.R.S. *Philosophical transactions of the Royal Society of London*, 1763.
- Lucas Berry and David Meger. Efficient epistemic uncertainty estimation in regression ensemble models using pairwise-distance estimators. *arXiv preprint arXiv:2308.13498*, 2023.
- Siddharth Betala, Samuel P Gleason, Ali Ramlaoui, Andy Xu, Georgia Channing, Daniel Levy, Cl  mentine Fourier, Nikita Kazeev, Chaitanya K Joshi, S  kou-Oumar Kaba, et al. Lemat-genbench: A unified evaluation framework for crystal generative models. *arXiv preprint arXiv:2512.04562*, 2025.
- Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight Uncertainty in Neural Networks. In *International conference on machine learning*, pages 1613–1622. PMLR, 2015.
- Ricky TQ Chen and Yaron Lipman. Flow matching on general geometries. *arXiv preprint arXiv:2302.03660*, 2024.
- Dario Coscia, Pim de Haan, and Max Welling. BLIPs: Bayesian Learned Interatomic Potentials. *arXiv preprint arXiv:2508.14022*, 2025a.
- Dario Coscia, Max Welling, Nicola Demo, and Gianluigi Rozza. Barnn: A bayesian autoregressive and recurrent neural network. *arXiv preprint arXiv:2501.18665*, 2025b.
- Thomas M Cover and Joy A Thomas. Network information theory. In *Elements of Information Theory*, pages 374–458. Wiley, 1st edition, 1991.
- Floor Eijkelboom, Grigory Bartosh, Christian Andersson Naesseth, Max Welling, and Jan-Willem van de Meent. Variational flow matching for graph generation. *Advances in Neural Information Processing Systems*, 37:11735–11764, 2024.
- Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. Deep ensembles: A Loss Landscape Perspective. *arXiv preprint arXiv:1912.02757*, 2019.
- Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International conference on machine learning*, pages 1050–1059, 2016.
- Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024.
- Alex Graves. Practical Variational Inference for Neural Networks. *Advances in Neural Information Processing Systems*, 24, 2011.
- Geoffrey E Hinton and Drew Van Camp. Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. In *Conference on Computational Learning Theory*, 1993.

- Pavel Izmailov, Sharad Vikram, Matthew D Hoffman, and Andrew Gordon Gordon Wilson. What are Bayesian Neural Network Posteriors Really Like? In *International conference on machine learning*, pages 4629–4640. PMLR, 2021.
- Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, Dan Gunter, David Skinner, Gerbrand Ceder, and Kristin A Persson. The materials project: A materials genome approach to accelerating materials innovation. *APL Materials*, 1(1):011002, 2013.
- Metod Jazbec, Eliot Wong-Toi, Guoxuan Xia, Dan Zhang, Eric Nalisnick, and Stephan Mandt. Generative uncertainty in diffusion models. *arXiv preprint arXiv:2502.20946*, 2025.
- Hamidreza Kamkari, Brendan Leigh Ross, Jesse C Cresswell, Anthony L Caterini, Rahul G Krishnan, and Gabriel Loaiza-Ganem. A geometric explanation of the likelihood ood detection paradox. *arXiv preprint arXiv:2403.18910*, 2024.
- Diederik P Kingma and Max Welling. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations*, 2014.
- Diederik P Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. *Advances in neural information processing systems*, 28, 2015.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- Benjamin Kurt Miller, Ricky TQ Chen, Anuroop Sriram, and Brandon M Wood. Flowmm: Generating materials with riemannian flow matching. *arXiv preprint arXiv:2406.04713*, 2024.
- Dmitry Molchanov, Arsenii Ashukha, and Dmitry Vetrov. Variational dropout sparsifies deep neural networks. *International Conference on Machine Learning*, 2017.
- Eric Nalisnick, Akihiro Matsukawa, Yee Whye Teh, Dilan Gorur, and Balaji Lakshminarayanan. Do deep generative models know what they don’t know? *arXiv preprint arXiv:1810.09136*, 2019.
- Theodore Papamarkou, Maria Skoularidou, Konstantina Palla, Laurence Aitchison, Julyan Arbel, David Dunson, Maurizio Filippone, Vincent Fortuin, Philipp Hennig, José Miguel Hernández-Lobato, et al. Position: Bayesian Deep Learning is Needed in the Age of Large-Scale AI. In *International Conference on Machine Learning*, 2024.
- Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *International Conference on Machine Learning*, volume 202, pages 28100–28127, 2023.
- Janis Postels, Francesco Ferroni, Huseyin Coskun, Nassir Navab, and Federico Tombari. Sampling-free epistemic uncertainty estimation using approximated variance propagation. *International Conference on Computer Vision*, 2019.
- Yiming Qin, Manuel Madeira, Dorina Thanou, and Pascal Frossard. Defog: Discrete flow matching for graph generation. *arXiv preprint arXiv:2410.04263*, 2024.
- Gabriel Raya and Luca Ambrogioni. Spontaneous symmetry breaking in generative diffusion models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Dejan Stančević and Luca Ambrogioni. The information dynamics of generative diffusion. *Entropy*, 28(2):195, 2026.
- Aik Rui Tan, Shingo Urata, Samuel Goldman, Johannes CB Dietschreit, and Rafael Gómez-Bombarelli. Single-model uncertainty quantification in neural network potentials does not consistently outperform model ensembles. *npj Computational Materials*, 9(1):225, 2023.

- Sida Wang and Christopher Manning. Fast dropout training. *International conference on machine learning*, pages 118–126, 2013.
- Andrew G Wilson and Pavel Izmailov. Bayesian Deep Learning and a Probabilistic Perspective of Generalization. *Advances in neural information processing systems*, 33:4697–4708, 2020.
- Brandon M Wood, Misko Dzamba, Xiang Fu, Meng Gao, Muhammed Shuaibi, Luis Barroso-Luque, Kareem Abdelmaqsoud, Vahe Gharakhanyan, John R Kitchin, Daniel S Levine, et al. UMA: A Family of Universal Models for Atoms. *arXiv preprint arXiv:2506.23971*, 2025.

FLOWING WITH CONFIDENCE APPENDIX

TABLE OF CONTENTS

A	Notation and Background	14
A.1	Notation	14
A.2	Continuous Transformations for Generative Modeling	14
A.3	Bayesian Neural Networks	15
B	Theoretical Results	16
C	Experimental Details and Hyperparameters	19
C.1	Datasets and Metrics	19
C.2	Model Architectures and Hyperparameters	20
D	Additional Results	21
D.1	Single-Layer vs Multi-Layer FMwC	21
D.2	Sampling Strategy Ablation	21
D.3	Full AUPRC Ablation	21
D.4	Propagation Variant Comparison	22
D.5	Endpoint vs Trajectory-Accumulated Scoring	22
D.6	Adaptive Stepping: Controllers	22
D.7	Hutchinson K -Sweep at Crystal Scale	23
E	Limitations and Future Work	23

A Notation and Background

This section provides a concise review of the relevant background, extends key concepts from prior work, and introduces the notation used throughout the paper.

A.1 Notation

We denote by $\mathbf{x} \in \mathbb{R}^d$ a data vector or state, with $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ drawn from the target empirical data distribution.

Maps or functions are generally denoted by Latin letters, such as u or f , while probabilities and distributions are represented by p or q . Neural network parameters are indicated using Greek letters: θ, ψ for deterministic parameters, and ω for stochastic parameters.

Conditional distributions are written explicitly, e.g., $p(\mathbf{x} \mid \mathbf{y})$ indicating the distribution of $\mathbf{x}$ condition on $\mathbf{y}$. Learned quantities, time-dependent or indexed quantities, are indicated by subscripts; for instance, v_θ denotes a function v learned with parameters θ , while $v_{t,\theta}$ indicates the function v_t indexed by t , with learned parameters θ .

A.2 Continuous Transformations for Generative Modeling

Generative models based on continuous transformations aim to learn a mapping that transports a simple reference distribution p_0 , such as a standard Gaussian, to a complex data distribution p_{data} . This is typically achieved by defining a parameterized transformation whose expressiveness is sufficient to capture the structure of the data, while remaining computationally tractable.

A general way to construct such transformations is via **time-dependent vector fields**. Let

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d \quad (7)$$

be a vector field. This induces a **flow map**

$$f : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d \quad (8)$$

defined as the solution to the **ordinary differential equation (ODE)**:

$$\frac{d}{dt} f_t(\mathbf{x}_0) = u_t(f_t(\mathbf{x}_0)), \quad f_0(\mathbf{x}_0) = \mathbf{x}_0. \quad (9)$$

Intuitively, $f_t(\mathbf{x}_0)$ tracks the trajectory of a point $\mathbf{x}_0$ as it moves along the vector field from $t = 0$ to $t = 1$. Under suitable smoothness conditions, this flow defines a smooth, invertible mapping that pushes the base distribution p_0 through a family of intermediate distributions $\{p_t\}_{t \in [0,1]}$, with p_1 approximating the target data distribution.

A.2.1 Distribution Evolution

The evolution of the distributions p_t along the flow is described by the **continuity equation**, a partial differential equation (PDE) that relates the vector field to the change in probability density:

$$\frac{\partial p_t(\mathbf{x})}{\partial t} + \nabla \cdot (p_t(\mathbf{x}) u_t(\mathbf{x})) = 0, \quad t \in [0, 1]. \quad (10)$$

This PDE guarantees that probability mass is conserved as it flows along the trajectories defined by u . In practice, however, the true transport vector field $u_t(\mathbf{x})$ is unknown and must be approximated. A common approach is to parametrize it using a neural network $v_{t,\theta}(\mathbf{x})$.

Traditional continuous normalizing flows (CNFs) train this network using a likelihood-based objective, which requires evaluating the **change-of-variables formula** along the flow:

$$\log p_1(f_1(\mathbf{x}_0)) = \log p_0(\mathbf{x}_0) - \int_0^1 \nabla \cdot v_{t,\theta}(f_t(\mathbf{x}_0)) dt, \quad (11)$$

where $\mathbf{x}_1 = f_1(\mathbf{x}_0)$. This requires repeatedly solving the ODE during training, which can be computationally expensive and sensitive to numerical errors.

A.2.2 Flow Matching

Instead of directly modeling densities, Flow Matching [Lipman et al., 2022] focuses on learning the vector field by matching **target velocities** at points sampled from the intermediate distributions. Specifically, given initial samples $\mathbf{x}_0 \sim p_0$ and target samples $\mathbf{x}_1 \sim p_1$, one defines a smooth interpolation $\mathbf{x}_t$ for $t \in [0, 1]$. This interpolation specifies a family of time-dependent distributions p_t and an associated velocity field $u_t(\mathbf{x}_t) = \frac{d\mathbf{x}_t}{dt}$ that satisfies the continuity equation.

Even though the true vector field $u_t(\mathbf{x}_t)$ and distributions p_t are unknown, one can construct a tractable **per-sample formulation** by defining **conditional flows** toward individual target datapoints $\mathbf{x}_1$. Concretely, for each $\mathbf{x}_1 \sim p_{\text{data}}$, we define a conditional trajectory

$$u_t(\mathbf{x} \mid \mathbf{x}_1), \quad (12)$$

which describes how a sample $\mathbf{x}_0 \sim p_0$ moves toward $\mathbf{x}_1$. The unconditional velocity field can then be expressed as a weighted average over these conditional trajectories:

$$u_t(\mathbf{x}) = \int u_t(\mathbf{x} \mid \mathbf{x}_1) \frac{p_t(\mathbf{x} \mid \mathbf{x}_1) p_{\text{data}}(\mathbf{x}_1)}{p_t(\mathbf{x})} d\mathbf{x}_1, \quad (13)$$

where $p_t(\mathbf{x} \mid \mathbf{x}_1)$ is the conditional distribution of $\mathbf{x}_t$ given the endpoint $\mathbf{x}_1$.

This formulation allows Flow Matching to define a well-posed regression target for the vector field even without explicit knowledge of the full vector field or intermediate densities. Training then reduces to matching the model velocity $v_{\theta}(\mathbf{x}_t, t)$ to the target conditional velocity along these trajectories, by minimizing:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_1 \sim p_{\text{data}}, t \sim \mathcal{U}[0,1], \mathbf{x}_t \sim p_t(\mathbf{x}_t \mid \mathbf{x}_1)} \left[\|v_{t,\theta}(\mathbf{x}_t) - u_t(\mathbf{x}_t \mid \mathbf{x}_1)\|^2 \right]. \quad (14)$$

A.3 Bayesian Neural Networks

Bayesian perspectives have played an important role in the development of deep learning methods, offering a coherent probabilistic framework for representing and reasoning about uncertainty [Hinton and Van Camp, 1993, Graves, 2011, Blundell et al., 2015, Gal and Ghahramani, 2016]. Instead of treating neural network parameters as fixed, Bayesian approaches consider the weights as random variables drawn from a prior distribution, $\omega \sim p(\omega)$. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ of N independent input–output pairs, the aim is to infer the posterior distribution over the weights, $p(\omega \mid \mathcal{D})$, using Bayes’ theorem [Bayes, 1763]. Exact computation of this posterior is generally infeasible for modern deep networks due to the high dimensionality and non-linearities of the parameter space. Consequently, various approximate inference methods have been proposed.

A.3.1 Deep Ensembles

Deep ensembles [Lakshminarayanan et al., 2017] approximate the posterior distribution by independently training multiple networks θ_i^{MAP} weights given different random initializations. The posterior is then approximated [Wilson and Izmailov, 2020] as a Dirac delta $p(\omega \mid \mathcal{D}) \approx p_{\text{ens}}(\omega \mid \mathcal{D}) = \sum_{i=1}^N \delta(\omega - \theta_i^{\text{MAP}})$, with δ the dirac-delta function. While they achieve state-of-the-art performance in many tasks [Fort et al., 2019, Tan et al., 2023], ensembles can be computationally expensive, particularly as the network size or the number of ensemble members grows. Each model in the ensemble requires a full set of parameters, leading to a linear increase in memory usage with the number of networks. This not only increases the storage requirements but also limits the ability to deploy ensembles in memory-constrained environments, such as edge devices or large-scale inference systems. Additionally, training multiple large networks is time-consuming, and inference requires aggregating predictions from all ensemble members, which can significantly slow down real-time applications. As a result, while deep ensembles provide robust uncertainty estimates, their scalability is inherently constrained by both model size and the number of ensemble components.

A.3.2 Variational-based Methods

Other widely used approaches include Variational Inference-based methods [Gal and Ghahramani, 2016, Blundell et al., 2015, Kingma et al., 2015], which replace the intractable posterior $p(\omega \mid \mathcal{D})$ with a tractable parametric distribution q_{ψ} that can be optimized efficiently over the parameters ψ

to resemble $p(\omega \mid \mathcal{D})$ by minimizing the evidence lower bound (ELBO). Despite the computational efficiency of these methods, they can be notoriously difficult to train, particularly for large-scale networks [Papamarkou et al., 2024]. Challenges include high sensitivity to hyperparameters, difficulties in balancing the trade-off between the likelihood and the KL divergence term in the ELBO [Blundell et al., 2015], and issues with poor posterior approximations for complex, high-dimensional weight spaces [Izmailov et al., 2021]. As a result, while variational approaches provide a scalable alternative to ensembles, achieving stable and accurate uncertainty estimates in deep networks remains a non-trivial problem.

A.3.3 Variational Adaptive Dropout

Coscia et al. [2025b] recently proposed Variational Adaptive Dropout (VAD), a Bayesian approach that presents the scalability properties of variational methods, with the accuracy gain typical of ensemble methods. In VAD, each weight ω_i in a neural network layer i is modelled with an *input-dependent variational distribution*, typically a Gaussian whose variance scales with the weight magnitude:

$$q_\psi(\omega_i \mid \mathbf{x}) = \mathcal{N}(\theta_i, \alpha_i(\mathbf{x})^2 \theta_i^2), \quad (15)$$

where θ_i is the mean parameter, and $\alpha_i(\mathbf{x})$ is a learned function of the input $\mathbf{x}$ controlling the adaptive dropout scale, usually given by an inference network E_γ . Sampling can be reparameterized as

$$\omega_i = \theta_i + \alpha_i(\mathbf{x}) \mid \theta_i \mid \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1), \quad (16)$$

which allows low-variance gradient estimates via the *local reparameterization trick* [Kingma et al., 2015]. For a linear layer with input $\mathbf{x}$, the resulting activation z_j is distributed as

$$z_j \sim \mathcal{N}\left(\sum_i x_i \theta_{ij}, \sum_i x_i^2 \alpha_i(\mathbf{x})^2 \theta_{ij}^2\right), \quad (17)$$

reducing the cost of sampling from per-weight to per-activation. Finally, adopting a zero-mean Gaussian prior with hyperparameter $p \in [0, 1]$:

$$\pi(\omega) = \mathcal{N}\left(\mathbf{0}, \frac{p}{1-p} \theta^2\right), \quad (18)$$

yields a closed-form KL divergence that depends only on the VAD coefficients:

$$\text{KL}[q_\psi(\omega \mid \mathbf{x}) \parallel \pi(\omega)] = \text{KL}(\{\alpha_i\}_i) = \sum_{i=1}^L \frac{(\alpha_i + 1)(1-p)}{p} + \log\left(\frac{p}{1-p}\right) - \log(\alpha_i) - 1. \quad (19)$$

This approach scales efficiently because the local reparameterization trick avoids sampling each weight individually, and the adaptive dropout coefficients $\alpha_i(\mathbf{x})$ can be predicted by a small auxiliary network E_γ , enabling input-dependent uncertainty without significantly increasing computational cost.

Variational Adaptive Dropout has been successfully applied to a variety of domains, including autoregressive and recurrent neural networks for molecule generation and PDE solving Coscia et al. [2025b], as well as graph neural networks for machine-learned interatomic potentials (MLIPs) Coscia et al. [2025a]. In this work, we employ Variational Adaptive Dropout extensively to parametrize the flow-matching vector field, enabling an efficient Bayesian flow-matching model.

B Theoretical Results

We begin by introducing a prior $\pi(v_t)$ over vector fields $v_t : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$. Given $\mathcal{D} = \{p_0, p_1\}$, our goal is to find the posterior $\pi(v_t \mid \mathcal{D})$. To do so, let us introduce a variational approximation $q_\psi(v_t)$, and minimize the Kullback-Leibler divergence between the variational and true posterior:

$$\text{KL}[q_\psi(v_t) \parallel \pi(v_t \mid \mathcal{D})] = \int \log \frac{dq_\psi}{d\pi}(v_t) dq_\psi(v_t), \quad (20)$$

where $\frac{dq_\psi}{d\pi}$ indicates the Radon-Nikodym derivative. Minimizing the divergence above is equivalent to maximizing:

$$\text{ELBO}(\psi) = \mathbb{E}_{v_t \sim q_\psi} [\log \mathcal{L}(\mathcal{D} \mid v_t)] - \text{KL}[q_\psi(v_t) \parallel \pi(v_t)], \quad (21)$$

where $\mathcal{L}(\mathcal{D} \mid v_t)$ is the functional likelihood over probability densities. To build a likelihood which penalizes deviations from a given target vector field that generates the probability path, we write:

$$\log \mathcal{L}(\mathcal{D} \mid v_t) = -\mathbb{E}_{t \sim U(0,1), \mathbf{x} \sim p_t(\mathbf{x})} \left[\|u_t(\mathbf{x}) - v_t(\mathbf{x})\|^2 \right]. \quad (22)$$

Note that the gradient of the log-likelihood is always equivalent to the gradient of the conditional Flow Matching objective (14), see [Lipman et al. \[2022\]](#) for reference.

Parameterizing function-space distributions. So far, we have reasoned about a variational posterior over vector fields v_t in an abstract, infinite-dimensional function space. While mathematically elegant, this formulation is not directly implementable: we cannot store or sample arbitrary functions. A natural solution is to *parameterize the vector field using a neural network* $v_{t,\omega}(\mathbf{x})$ with random parameters ω . We can then link the variational distribution over the network parameters to the functions directly by:

$$q_\psi(v_t) = \int \delta(v_t - v_{t,\omega}) q_\psi(\omega) d\omega. \quad (23)$$

Intuitively, this says: to sample a vector field v_t , first sample parameters $\omega \sim q_\psi(\omega)$, then evaluate the corresponding function (network) $v_{t,\omega}$ given the random parameters ω . Given the parametrization of equation (23) and the objective in equation (21) we obtain the following proposition:

Proposition B.1 (Tractable parameter-space ELBO). *Let $v_{t,\omega}$ be a parametrization of the vector field with parameters ω , and let $q_\psi(\omega)$ denote a variational posterior over ω . Then, given equation (23), the function-space ELBO in equation (21) reduces to the following tractable parameter-space objective:*

$$\text{ELBO}(\psi) = \mathbb{E}_{\omega \sim q_\psi(\omega)} [\log \mathcal{L}(\mathcal{D} \mid \omega)] - \text{KL}[q_\psi(\omega) \parallel \pi(\omega)], \quad (24)$$

where the likelihood evaluates the discrepancy between the true and parametrized vector fields:

$$\log \mathcal{L}(\mathcal{D} \mid \omega) = -\mathbb{E}_{t \sim U(0,1), \mathbf{x} \sim p_t(\mathbf{x})} \left[\|u_t(\mathbf{x}) - v_{t,\omega}(\mathbf{x})\|^2 \right]. \quad (25)$$

Proof. Let $\mathcal{V}$ denote the function space of admissible vector fields. By definition, the variational distribution over vector fields is expressed as

$$q_\psi(v_t) = \int \delta(v_t - v_{t,\omega}) q_\psi(\omega) d\omega, \quad (26)$$

where δ is the Dirac delta functional in the function space $\mathcal{V}$, and $\omega \in \Omega$ parameter space. Let $T : \Omega \rightarrow \mathcal{V}$ be the measurable map defined by

$$T(\omega) := v_{t,\omega}. \quad (27)$$

By construction, the variational distribution over vector fields is the push-forward measure

$$q_\psi(v_t) = T_\# q_\psi(\omega). \quad (28)$$

Consider first the expected log-likelihood term. By the definition of the push-forward measure:

$$\int_{\mathcal{V}} \log \mathcal{L}(\mathcal{D} \mid v_t) dq_\psi(v_t) = \int_{\Omega} \log \mathcal{L}(\mathcal{D} \mid T(\omega)) dq_\psi(\omega) = \mathbb{E}_{\omega \sim q_\psi(\omega)} [\log \mathcal{L}(\mathcal{D} \mid \omega)]. \quad (29)$$

Next, assume that the prior over vector fields $\pi(v_t)$ is also induced by a prior $\pi(\omega)$ on parameters through the same mapping T , i.e. $\pi(v_t) = T_\# \pi(\omega)$. Since T is deterministic and measurable, the data-processing inequality for the Kullback–Leibler divergence [[Cover and Thomas, 1991](#)] under pushforward gives

$$\text{KL}[q_\psi(v_t) \parallel \pi(v_t)] = \text{KL}[T_\# q_\psi(\omega) \parallel T_\# \pi(\omega)] \leq \text{KL}[q_\psi(\omega) \parallel \pi(\omega)], \quad (30)$$

with equality when T is injective. Optimizing the parameter-space objective therefore maximizes a valid lower bound on the functional ELBO, which is tight under injectivity of the parameterization and otherwise tight up to the usual weight-space symmetries (permutations and scalings) of $v_{t,\omega}$.

Combining the two terms, the functional-space ELBO is lower-bounded by

$$\text{ELBO}(\psi) \geq \mathbb{E}_{\omega \sim q_\psi(\omega)} [\log \mathcal{L}(\mathcal{D} \mid \omega)] - \text{KL}[q_\psi(\omega) \parallel \pi(\omega)], \quad (31)$$

which is the tractable parameter-space objective we optimise. $\square$

Observation B.1 (Bayesian model averaging preserves valid flows). Let $q_\psi(v_t)$ be a posterior distribution over vector fields v_t such that each sample v_t satisfies the continuity equation

$$\partial_t p_t + \nabla \cdot (p_t v_t) = 0. \quad (32)$$

Then, the posterior mean vector field

$$\bar{v}_t(\mathbf{x}) := \mathbb{E}_{v_t \sim q_\psi} [v_t(\mathbf{x})] \quad (33)$$

also satisfies the continuity equation, i.e.

$$\partial_t p_t + \nabla \cdot (p_t \bar{v}_t) = 0. \quad (34)$$

Proof. By assumption, for each v_t in the support of q_ψ , the continuity equation $\partial_t p_t + \nabla \cdot (p_t v_t) = 0$ holds pointwise in $(t, \mathbf{x})$. Integrating against q_ψ and exchanging integration with ∂_t and $\nabla \cdot$ by linearity yields

$$0 = \int (\partial_t p_t + \nabla \cdot (p_t v_t)) dq_\psi(v_t) = \partial_t p_t + \nabla \cdot \left(p_t \int v_t dq_\psi(v_t) \right) = \partial_t p_t + \nabla \cdot (p_t \bar{v}_t). \quad (35)$$

Bayesian averaging of valid vector fields therefore preserves the validity of the probability flow. $\square$

Observation B.2 (Equivalence of Expected Likelihood and Flow Matching Gradients). Given a reparameterization of $q_\psi(\omega)$ such that $\omega = g_\psi(\epsilon)$ with $\epsilon \sim p(\epsilon)$, the Bayesian expected likelihood gradient satisfies

$$\begin{aligned} & -\nabla_\psi \mathbb{E}_{\omega \sim q_\psi(\omega)} [\log \mathcal{L}(\mathcal{D} \mid \omega)] \\ &= \nabla_\psi \mathbb{E}_{\omega \sim q_\psi(\omega), \mathbf{x}_1 \sim p_{\text{data}}, t \sim \mathcal{U}[0,1], \mathbf{x}_t \sim p_t(\mathbf{x} \mid \mathbf{x}_1)} [\|v_{t,\omega}(\mathbf{x}_t) - u_t(\mathbf{x}_t \mid \mathbf{x}_1)\|^2]. \end{aligned} \quad (36)$$

Proof. We proceed in two steps:

Step 1: Deterministic equivalence. For a deterministic parameter θ , Lipman et al. [2022] showed:

$$\begin{aligned} & \nabla_\theta \mathbb{E}_{t \sim U(0,1), \mathbf{x} \sim p_t(\mathbf{x})} [\|u_t(\mathbf{x}) - v_{t,\theta}(\mathbf{x})\|^2] \\ &= \nabla_\theta \mathbb{E}_{\mathbf{x}_1 \sim p_{\text{data}}, t \sim U(0,1), \mathbf{x}_t \sim p_t(\mathbf{x} \mid \mathbf{x}_1)} [\|v_{t,\theta}(\mathbf{x}_t) - u_t(\mathbf{x}_t \mid \mathbf{x}_1)\|^2]. \end{aligned} \quad (37)$$

This establishes equivalence in the deterministic case.

Step 2: Bayesian lifting via reparameterization. Suppose $\omega \sim q_\psi(\omega)$ admits a reparameterization $\omega = g_\psi(\epsilon)$ with $\epsilon \sim p(\epsilon)$. Then, for any function $f(\omega)$, the reparameterization trick [Kingma and Welling, 2014] holds:

$$\mathbb{E}_{\omega \sim q_\psi(\omega)} [f(\omega)] = \mathbb{E}_{\epsilon \sim p(\epsilon)} [f(g_\psi(\epsilon))], \quad \nabla_\psi \mathbb{E}_{\omega \sim q_\psi(\omega)} [f(\omega)] = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\nabla_\psi f(g_\psi(\epsilon))]. \quad (38)$$

Applying this to the deterministic equivalence in Step 1, we obtain

$$\begin{aligned} \nabla_\psi \mathbb{E}_{\omega \sim q_\psi(\omega)} \mathbb{E}_{t,\mathbf{x}} [\|u_t(\mathbf{x}) - v_{t,\omega}(\mathbf{x})\|^2] &= \mathbb{E}_{\epsilon \sim p(\epsilon)} \nabla_\psi \mathbb{E}_{t,\mathbf{x}} [\|u_t(\mathbf{x}) - v_{t,g_\psi(\epsilon)}(\mathbf{x})\|^2] \\ &= \mathbb{E}_{\epsilon \sim p(\epsilon)} \nabla_\psi \mathbb{E}_{\mathbf{x}_1, t, \mathbf{x}_t \mid \mathbf{x}_1} [\|v_{t,g_\psi(\epsilon)}(\mathbf{x}_t) - u_t(\mathbf{x}_t \mid \mathbf{x}_1)\|^2] \\ &= \nabla_\psi \mathbb{E}_{\omega \sim q_\psi(\omega), \mathbf{x}_1, t, \mathbf{x}_t} [\|v_{t,\omega}(\mathbf{x}_t) - u_t(\mathbf{x}_t \mid \mathbf{x}_1)\|^2]. \end{aligned} \quad (39)$$

Identifying the left-hand side with the negative expected log-likelihood gradient under q_ψ , we conclude the desired equivalence. $\square$

Variational Adaptive Dropout Flow Matching Objective. Building on Observation B.2, we can instantiate the reparameterization $\omega = g_\psi(\epsilon)$ with Variational Adaptive Dropout. In VAD, each weight ω_i is modeled as an input-dependent Gaussian:

$$q_\psi(\omega_i \mid \mathbf{x}) = \mathcal{N}(\theta_i, \alpha_i(\mathbf{x})^2 \theta_i^2), \quad (40)$$

where θ_i is the mean parameter and $\alpha_i(\mathbf{x})$ is predicted by a small auxiliary network $E_\gamma(\mathbf{x})$ that controls the adaptive dropout scale. This admits a simple reparameterization:

$$\omega_i = g_{\psi}(\mathbf{x}, \epsilon_i) = \theta_i + \alpha_i(\mathbf{x}) |\theta_i| \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1). \quad (41)$$

Substituting the VAD reparameterization into the Bayesian Flow Matching gradient, we obtain the training objective:

$$\text{ELBO}(\psi) = -\mathbb{E}_{\mathbf{x}_1, t, \mathbf{x}_t, \epsilon} \left[\left\| v_{t, g_{\psi}(\mathbf{x}_1, \epsilon)}(\mathbf{x}_t) - u_t(\mathbf{x}_t | \mathbf{x}_1) \right\|^2 \right] - \text{KL}(\{\alpha_i\}_{i \geq 1}). \quad (42)$$

With, $\mathbf{x}_1 \sim p_{\text{data}}$, $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_t \sim p_t(\mathbf{x} | \mathbf{x}_1)$, $\epsilon \sim \mathcal{N}(0, I)$.

Here:

- $g_{\psi}(\mathbf{x}_1, \epsilon)$ implements the VAD reparameterization (41), with input-dependent adaptive dropout scales $\alpha_i(\mathbf{x}_1)$.
- Sampling from ϵ implements the stochasticity of the variational posterior q_{ψ} .
- The Kullback Leibler term only depends on the adaptive dropout coefficients, as implemented in [Coscia et al. \[2025a\]](#).

C Experimental Details and Hyperparameters

In this section we report datasets, evaluation metrics, and architectural hyperparameters used for the main-text experiments. All models were trained on NVIDIA L4 GPUs (24GB), using the validation set for checkpoint selection and the test set for evaluation.

C.1 Datasets and Metrics

We evaluate on three datasets spanning a 2D synthetic target, an image domain, and a structured scientific domain. Reported numbers are averaged over three random seeds unless stated otherwise.

Checkerboard. A 2D synthetic target whose support is the eight black cells of a 4×4 chessboard tiling of $[-2, 2]^2$, with uniform density on the support and zero outside. The base distribution is $p_0 = \mathcal{N}(0, I_2)$. We train on 5×10^5 samples from the analytical target and evaluate on 10^4 generated samples. We report three metrics. *Misplacement* is the fraction of generated samples falling outside the eight-cell support, evaluated by direct geometric containment (lower is better; the random baseline is 50%). *KL* is the symmetric Kullback–Leibler divergence between a Gaussian-kernel KDE of the generated samples (bandwidth chosen by Silverman’s rule) and the analytical target on a 200×200 grid (lower is better). *Filtering AUPRC* ($\uparrow$) evaluates confidence-guided retention as a binary classification: per-sample confidence is the predictor, and a binary label flagging samples we want to discard is the target; we sweep a retention threshold from "keep nothing" to "keep everything" and integrate precision against recall on the positive class. AUPRC is bounded above by 1 and below by the positive-class prevalence π (the precision of random retention).

MNIST. We use the standard MNIST training split as the target distribution and the held-out test split for downstream classifier evaluation. Images are normalised to $[0, 1]$ and treated as 784-dimensional Euclidean vectors; the base distribution is $p_0 = \mathcal{N}(0, I_{784})$. We train on the 6×10^4 training images and evaluate on 10^4 generated images per run. We report two metrics. *Classifier-derived FID* ($\downarrow$) is the Fréchet distance between the penultimate-layer activations of a LeNet-style classifier pretrained on MNIST, evaluated on the generated batch and on a held-out reference batch from the test split. *Filtering AUPRC* ($\uparrow$, defined as in **Checkerboard**) uses *classifier failure* as the positive class: the pretrained classifier’s top-1 prediction on the generated image disagreeing with the requested class label.

Crystal. We use the FlowMM [\[Miller et al., 2024\]](#) backbone for de novo inorganic crystal generation, trained on the Materials Project [\[Jain et al., 2013\]](#) subset of stable structures used in the original FlowMM release. The flow operates on the Riemannian product manifold of fractional coordinates ($[0, 1]^{3N}$ with periodic identifications), lattice parameters ($\mathbb{R}^6$ via the Niggli-reduced parameterisation), and atomic species (categorical, embedded in the simplex). We evaluate 2500 generated structures per run. Quality is reported on *LeMat-GenBench* [\[Betala et al., 2025\]](#) after

post-relaxation by a UMA [Wood et al., 2025] single-point evaluation on the raw CIF: *Meta%* ($\uparrow$) is the fraction of generated structures with $e_{\text{above hull}} < 0.1$ eV/atom (metastable), *Stable%* ($\uparrow$) the fraction at $e_{\text{above hull}} < 0$ eV/atom, and $\bar{e}_{\text{above hull}}$ ($\downarrow$) the mean energy above the convex hull in eV/atom. *Filtering AUPRC* ($\uparrow$, defined under **Checkerboard**) uses *generation success* as the positive class: $e_{\text{above hull}} < 0.1$ eV/atom. Due to the computational complexity only a single seed run was reported.

C.2 Model Architectures and Hyperparameters

Checkerboard. The deterministic backbone is an MLP with SiLU activations and a sinusoidal time embedding concatenated to the input at every layer; widths and depths are listed in Tab. 5. FMwC attaches an inference network E_γ to every linear layer of the backbone. Each E_γ is a small two-layer MLP that takes the layer’s mean activation μ_l and the time t as input and outputs the per-channel log-dropout scale $\log \alpha_l(x, t)$, which parameterises the Gaussian posterior in Eq. 3; the inference networks share their architecture across layers but not their weights. The baselines reuse the same backbone: *FM* drops the inference networks entirely, *MC-Dropout* replaces them with standard fixed dropout at rate p , and *Ensemble* is k independently trained copies of the FM backbone with different random seeds.

MNIST. The MNIST backbone is a UNet with sinusoidal time embedding broadcast to every residual block; the base channel count and class-embedding dimension are listed in Tab. 5. Class conditioning, when used, is provided as a learned class-embedding vector added to the time embedding before the residual broadcast. Inference networks E_γ are attached to every convolutional and linear layer of the UNet (downsampling stages, bottleneck, upsampling stages), with the same two-layer-MLP construction used on Checkerboard; each E_γ takes the spatially-pooled mean activation of its layer together with the time embedding as input and emits a per-channel log-dropout scale, so VAD noise is shared across spatial positions within a feature map but varies between channels. Baselines are constructed as in Checkerboard, with $k=5$ independently retrained UNet copies for the Ensemble.

Crystal. The deterministic backbone is the FlowMM equivariant graph network of Miller et al. [2024], used unchanged from the original architecture; we refer the reader to that paper for the message-passing structure, the manifold-aware update rules, and the parameter count. FMwC attaches an inference network to every linear layer in the message-passing and readout blocks, with the same two-layer-MLP construction used on the Euclidean applications. The deterministic-FM baseline is the FlowMM checkpoint released with Miller et al. [2024], retrained under our infrastructure but with the same hyperparameters. Disagreement-based baselines (MC-Dropout, Ensemble) are not reported on the Crystal column because $k \times$ retraining of FlowMM and $k \times$ stochastic trajectories per generated structure are both prohibitive at the throughput LeMat-GenBench [Betala et al., 2025] evaluates against.

Table 5: **Network architecture hyperparameters across the three applications.** FMwC attaches an inference network E_γ to every linear layer of the backbone. MC-Dropout shares the backbone and replaces the inference networks with fixed dropout at rate p ; Ensemble is k independent copies of the FM backbone. Dashes in the Configuration row mark a backbone used unchanged from Miller et al. [2024]; dashes in the MC-Dropout and Ensemble rows mark baselines not reported on the Crystal column.

	Checkerboard	MNIST	Crystal
Backbone	MLP	UNet	CSPNet
Configuration	hidden 512, depth 3	base ch. 32, cond. dim 128	hidden 512, time-emb. 256, depth 6
Inf. net width	64	64	64
Inf. net depth	2	2	2
MC-Dropout p	0.1	0.1	—
Ensemble k	5	5	—

Table 6: **Training hyperparameters across the three applications.** All FMwC runs optimise the parameter-space ELBO of Eq. 1 with KL weight β scaling the closed-form per-layer KL between the VAD posterior $\mathcal{N}(\theta_t, \alpha_t^2 \theta_t^2)$ and a Gaussian prior at fixed rate α_p ; the KL term is annealed cyclically with a linear warmup. Dashes mark settings not used on the application.

	Checkerboard	MNIST	Crystal
Optimiser	Adam	Adam	AdamW
Learning rate	10^{-3}	10^{-4}	5×10^{-4}
Weight decay	—	—	5×10^{-3}
Batch size	4096	256	256
Training length	20,000 iter.	50,000 iter.	2,000 epochs
LR schedule	—	—	cosine to 10^{-5}
EMA decay	—	—	0.999
KL weight β	10^{-3}	10^{-3}	10^{-3}
Prior rate α_p	0.3	0.3	0.1
KL cycles	4	4	4
KL warmup fraction	0.1	0.1	0.1

D Additional Results

D.1 Single-Layer vs Multi-Layer FMwC

We compare two FMwC parameterisations on the Checkerboard. *Output-only* FMwC injects VAD noise into the final linear layer alone. *Multi-layer* FMwC, the version used in the main text, attaches VAD to every linear layer along the depth. Tab. 7 reports misplacement (fraction of generated points outside the eight-cell support) and KL between a kernel-density estimate of generated samples and the analytical target. The two parameterisations are within 0.3 percentage points of each other on misplacement, so output-only FMwC is not a quality regression. The cost of the simplification is on the confidence side: only multi-layer propagation accumulates the variance signal that the temporal-ratio score relies on (Tab. 9).

Table 7: **Output-only FMwC matches multi-layer FMwC on Checkerboard quality.** Misplacement and KL on the Checkerboard, both lower-is-better, under MAP integration with $N=50$ Euler steps. The AUPRC companion that breaks the tie is Tab. 9.

Method	Mispl.% ↓	KL ↓
Output-FMwC MAP	4.3%	0.088
FMwC MAP	4.6%	0.084

D.2 Sampling Strategy Ablation

The variational posterior over weights admits three natural decoders for sample generation. (1) *Stochastic*: draw one weight sample $\omega \sim q_\psi(\omega | x, t)$ at each ODE step and integrate that single trajectory. (2) *Mean velocity* ($k=5$): draw k weight samples per step, average their velocity outputs, and integrate the mean. (3) *MAP*: integrate the deterministic mean field $\mu_t(x)$ produced by removing stochasticity in the network, which is what Eq. 4 produces alongside the variance trajectory at no extra cost. Tab. 8 reports Checkerboard misplacement under all three for both FMwC parameterisations. The three perform almost exactly, so we adopt MAP throughout the paper: it is the cheapest of the three, and it is exactly the trajectory the variance recursion is already running on.

D.3 Full AUPRC Ablation

Tab. 9 is the unified Checkerboard AUPRC table across every method and every readout we tested. Positive class is misplacement (a generated point falling outside the eight-cell support); the FM row is the positive-class rate, which is the random-retention floor. Two facts read off the table. FMwC’s temporal-confidence ratio at one trajectory (AUPRC=0.55) sits within 0.05 of the $k=5$ FMwC-MC (0.58), but only integrating a single trajectory.

Table 8: **Stochastic, mean-velocity ($k=5$), and MAP decoders are within noise on Checkerboard quality.** Misplacement under each decoder for output-only and multi-layer FMwC at $N=50$ Euler steps; MAP is the default elsewhere in the paper.

Method	Stochastic ↓	Mean vel ($k=5$) ↓	MAP ↓
Output-FMwC	4.3%	4.3%	4.3%
FMwC	4.6%	4.6%	4.6%

Table 9: **FMwC’s temporal-ratio score reaches the $k=5$ between-model AUPRC at one-fifth the trajectories on the Checkerboard.** Positive class is misplacement; FM row is the positive-class rate. Trajectories column is forward integrations per generated sample.

Method	Scoring	AUPRC ↑	Traj.
FM	random	0.06	1
FMwC ($k=5$)	MC endpoint spread	0.58	5
FMwC	temporal confidence	0.55	1
FMwC	endpoint var	0.44	1
FMwC	traj-integrated	0.33	1
FMwC	analytical endpoint	0.30	1

D.4 Propagation Variant Comparison

The per-layer recursion in Eq. 4 requires propagating (μ, σ^2) through pointwise nonlinearities ϕ , which means evaluating $\mathbb{E}[\phi(z)]$ and $\text{Var}[\phi(z)]$ for $z \sim \mathcal{N}(\mu, \sigma^2)$. We compare three closures: the first-order Taylor expansion of Wang and Manning [2013], its second-order extension, and 10-node Gauss–Hermite quadrature [Postels et al., 2019]. Tab. 10 reports endpoint AUPRC on the Checkerboard for the three. They are identical to two decimal places, so the choice does not matter at the depths used in this paper; we default to Gauss–Hermite as the most numerically stable of the three under wider activations.

D.5 Endpoint vs Trajectory-Accumulated Scoring

The two readouts c_{ep} and c_{tc} defined in Sec. 3.1 differ in what they aggregate: c_{ep} is the analytical variance at the integration endpoint $t=1$, while c_{tc} is the per-component late-window ratio of the variance trajectory. Tab. 11 compares the two on the Checkerboard. Trajectory-integrated scoring lifts AUPRC from 0.30 to 0.33 for the analytical readout at one trajectory: confidence has temporal dynamics that endpoint-only scoring averages over. The Monte-Carlo variant at $k=5$ reaches 0.58 at the endpoint, which is the upper bound at this trajectory budget but requires five integrations per sample.

D.6 Adaptive Stepping: Controllers

Both controllers run on top of the FMwC variance trajectory at one forward pass per integrator step on a fixed budget of N NFE per sample. Index samples by j , steps by $i \in \{0, \dots, N-1\}$; write t_j for the current per-sample time at step i and $K_j = N - i$ for the steps remaining. The per-sample scalar trajectory norm of σ_t^2 is modality-specific: $\sigma_j = (\sum_d \sigma_{t,d}^2)^{1/2}$ for Checkerboard (sum over the two Euclidean dimensions), $\sigma_j = \frac{1}{CHW} \sum_{c,h,w} \sqrt{\sigma_{t,c,h,w}^2}$ for MNIST (mean of the pixel-wise std over channels and pixels), and $\sigma_j = \sigma_{\text{coord},j} + \sigma_{\text{lattice},j} + \sigma_{\text{type},j}$ for the crystal flow (coord and type summed over atoms, lattice as the Frobenius norm of its 3×3 variance matrix). Its EMA at rate $\lambda \in (0, 1]$ is $\bar{\sigma}_j \leftarrow (1 - \lambda) \bar{\sigma}_j + \lambda \sigma_j(t_j)$, initialised to $\sigma_j(0)$.

The *naive EMA controller* turns $(\bar{\sigma}_j, \sigma_j)$ into a per-sample step $\Delta t_j \leftarrow \frac{1-t_j}{K_j} (\bar{\sigma}_j / \sigma_j(t_j))^b$, clipped to $[\Delta t_{\min}, 1 - t_j]$ with $\Delta t_{\min} = 10^{-2}/N$ and forced to $1 - t_j$ on the last step so every sample lands at $t=1$ in exactly N NFE; $b \geq 0$ is the boost factor ($b=0$ recovers uniform). Because $\bar{\sigma}_j$ peaks early on every modality, this rule allocates compute to the early window — well-aligned for the Euclidean toys but not for the crystal flow, where the window that governs sample quality is late.

Table 10: **First-order Taylor, second-order Taylor, and Gauss-Hermite are indistinguishable at the depths used in this paper.** Endpoint AUPRC on the Checkerboard for the three nonlinearity closures inside the layer-to-layer variance recursion of Eq. 4.

Propagation	Endpoint AUPRC $\uparrow$
1st-order	0.30
2nd-order	0.30
Gauss-Hermite	0.30

Table 11: **Trajectory-accumulated scoring lifts FMwC’s analytical AUPRC by +0.03 at one trajectory.** Endpoint vs. trajectory-integrated AUPRC on the Checkerboard for the analytical variance estimators.

Method	Endpoint AUPRC $\uparrow$	Traj-int AUPRC $\uparrow$
FMwC	0.30	0.33

FMwC Online Adaptive, deployed across all three modalities, replaces the scalar trajectory norm with a per-component decomposition and gates the rule on a late-only window. Splitting σ_t^2 into per-component variances $\sigma_{c,j}^2(t_j)$ with c ranging over the modality’s components (Euclidean axes for Checkerboard, image channels for MNIST, {coord, lattice} for the crystal flow), and tracking per-component EMAs $\bar{\sigma}_{c,j}$ (rate λ , init $\sigma_{c,j}(0)$), the excess ratio $r_{c,j}(t_j) = \max(\sigma_{c,j}(t_j)/\bar{\sigma}_{c,j} - 1, 0)$ fires only on components currently above their own running mean. The grid stays uniform $\Delta t = 1/N$; for $t_j \geq t_{\text{late}}$ the velocity slice v_c is damped as $v_c \leftarrow (1 - \kappa_c r_{c,j}(t_j))_+ v_c$ with $\kappa_c \geq 0$ a per-component damping gain and $(\cdot)_+$ the ReLU clip, and is left unchanged for $t_j < t_{\text{late}}$. On the FlowMM crystal flow this composes with the FlowMM anti-annealing factor $1 + s_0 t$ so the effective per-component slope is $s_{c,j} = s_0 (1 - \kappa_c r_{c,j}(t_j))_+$ with s_0 the base slope shared with FM Uniform. The two ingredients that recover the gain the naive scalar EMA loses on crystals are the late-only gate (compute is not spent on the early peak in $\bar{\sigma}_j$) and the per-component split (damping responds to the component that is actually rising at t_j).

D.7 Hutchinson K -Sweep at Crystal Scale

Tab. 4 in the main text and Fig. 5 both use a high- K Hutchinson stochastic-trace estimator of $|\nabla \cdot v_{t,\theta}|$ as the divergence reference on the FlowMM crystal backbone [Miller et al., 2024], since the exact divergence requires D backward passes per step at the model dimension and is not directly accessible at this scale. Tab. 12 reports the full K -sweep, $K \in \{1, 5, 10, 25, 50, 100, 250, 500\}$, in two regimes: per-step (correlation evaluated at each ODE timestep, then averaged across timesteps) and integrated (correlation between trajectory-summed quantities, one scalar per sample). The trajectory-integrated correlation is essentially flat in K (ρ rises from 0.947 at $K=1$ to 0.955 from $K=5$ onwards), while the per-step correlation continues to climb and only saturates around $K \approx 100$ ($\rho=0.853$, vs. 0.500 at $K=1$). This is the empirical basis for the matched-FLOPs claim in Sec. 4 that one forward pass of FMwC corresponds to Hutchinson at $K \approx 7$ probes on a per-step basis.

E Limitations and Future Work

While FMwC delivers calibrated, actionable confidence scores across synthetic, image, and scientific domains, several limitations remain that we outline below to guide future work.

Our Crystal evaluation is reported on a single seed due to the cost of UMA [Wood et al., 2025] post-relaxation and LeMat-GenBench [Betala et al., 2025] throughput; trends are consistent with the multi-seed Checkerboard and MNIST results, but a full sweep would tighten variance estimates. More broadly, our benchmarks span a 2D target, an image domain, and a manifold-valued scientific domain, but do not yet probe text, video, or large-scale molecular systems.

On the theoretical side, while our variance decomposition identifies *when* a sample is uncertain, a formal link between the posterior divergence along a trajectory and the resulting sample-level

Table 12: **Per-step Hutchinson correlation climbs to saturation around $K \approx 100$; trajectory-integrated correlation is saturated from $K=5$.** Spearman ρ and Pearson r between Hutchinson at K probes and the high- K reference on the FlowMM crystal backbone, in per-step and integrated regimes.

K	Per-step ρ	Per-step r	Integrated ρ	Integrated r
1	0.500	0.410	0.947	0.919
5	0.682	0.610	0.955	0.933
10	0.745	0.674	0.955	0.934
25	0.807	0.733	0.955	0.934
50	0.835	0.762	0.955	0.934
100	0.853	0.779	0.955	0.934
250	0.864	0.790	0.955	0.934
500	0.868	0.794	0.955	0.934

confidence is still missing. Establishing such a connection would put confidence-guided filtering on firmer footing and clarify which segments of a flow contribute most to epistemic uncertainty.

Our use of confidence is also primarily diagnostic. A natural next step is to study *how* to act on uncertainty at inference time, through adaptive ODE solvers that refine integration where posterior variance is high, guidance schemes steering trajectories away from low-confidence regions, or selective re-sampling that allocates compute to the hardest trajectories.